



FOUR LAYERS OF AI:
LLMS, AGENTS, SKILLS, AND MCPS
CONTINUOUS IMPROVEMENT

** INTERNAL USE ONLY **

TABLE OF CONTENTS

HOUSEKEEPING.....3

START HERE: YOUR FIRST CHOICE3

OUTCOME3

FOUR LAYERS OF AI.....4

 LLM.....4

 AGENT5

 APPLY IT: LLM OR AGENT?6

 SKILL.....6

 MCP7

 APPLY IT: SKILL OR MCP?8

 CHOOSING THE RIGHT LAYER8

 GUIDED PLAYGROUND9

 REFLECTION 10

FINAL THOUGHTS..... 10

PLUGIN TIPS 10

INTERNAL USE ONLY

FOUR LAYERS OF AI: LLMS, AGENTS, SKILLS, AND MCPS

OVERVIEW

In this workshop, you will explore the four layers of UWM's AI tool stack: LLMs, Agents, Skills, and MCPs. You will compare their purpose, when each layer is useful, and how tool choice can affect token consumption and output quality.

HOUSEKEEPING

Any third-party package (or library) must be brought to the Technology Review Committee (TRC) for review before downloading. Your model must be set to auto in GitHub Copilot.

**THIRD-PARTY
PACKAGES**

**GITHUB COPILOT
SETTINGS**

START HERE: YOUR FIRST CHOICE

Choose one task you might use AI for in your role. What tool or approach would you reach for first and why? (e.g., M365 Copilot, GitHub Copilot, Agents, Skills, MCPs)

Prompt	Your Response
What task might you use AI for?	Development-related activities
Which layer, tool, or approach would you use first?	GitHub Copilot via "Ask/Auto" setup (generally no Agent)
Why would you start there?	Has ability, if need to, to switch to deep-dive, highly analytical models (via "Agent/xxx" selection)

OUTCOME

By the end of today's training, you will accomplish the following:

- Demonstrate basic use of an LLM, Agent, Skill, and MCP.

FOUR LAYERS OF AI

Below, you will find four tables that outline each layer of AI used at UWM. Each table contains what the layer is, it’s key capabilities, when it’s best to use it, and its estimated token consumption so that you can be more intentional about which layer you choose for your needs.

TIP: start with the **simplest** layer that can do the job and only move up when the task requires more capability.

LLM

A Large Language Model (LLM) is the foundational AI layer behind tools like Copilot. This is what most of us think of when we think of chatting with Copilot. You provide a prompt and any needed context, and the model generates a response. It’s the foundation of all layers - you cannot use an Agent, Skill, or MCP without the LLM.

Category	Details
Key Capabilities	Asking questions, drafting a message, reviewing an email, summarizing notes, explaining code, or brainstorming ideas – anything that <i>doesn’t require you to take action</i> . Works best with information the learner provides directly. The LLM only has knowledge of what it’s trained on.
Concrete Example	<i>Can you review my email for clarity, grammar, and conciseness?</i>
Best Used When	The task is quick, text-based, exploratory, or does not require live system access. When a direct prompt is enough to accomplish your goal.
Don’t Use When	You need live data, connected systems, repeatable workflow controls, or deeper repo/context awareness. Your goal is to have something completed. LLMs are designed to generate information, not take action.
Token Severity	Low
What Drives Token Use	Prompt, response, chat history, and any included context or attachments.
AI Credit Takeaway	Lowest cost. Lighter models may process more tokens per credit; e.g., GPT-5 Mini averages ~5,000 tokens/credit vs Claude Sonnet ~1,000.

NOTES:

AGENT

An Agent is an AI-powered persona that can work toward a broader goal. Copilot can determine which agent is best to use for your provided prompt/task at hand, or you can specify which agent you would like it to use. Access the list of available agents with the following command: `/agent`.

Category	Details
Key Capabilities	Break down work into steps, decide which tools or skills to use, and continue working toward a defined outcome – <i>anything where action is needed</i> , not just information, and when the work is investigative, multi-step, or tool-dependent.
Concrete Example	Use the <code>[bLank]</code> agent to pull Epic details from Jira, search the codebase for relevant patterns and file locations, and identify architectural constraints and existing solutions.
Best Used When	The task requires any number of actions to be taken that goes beyond a simple prompt.
Don't Use When	A simple summary, rewrite, or one-off question would be enough.
Token Severity	Variable
What Drives Token Use	Reasoning, workflow steps, tool calls, available context, and how the agent is designed. Well-defined rules and workflows can improve efficiency and reduce unnecessary token usage.
AI Credit Takeaway	Variable cost. Well-designed agents can conserve tokens by following structured rules and workflows. More complex goals, additional context, or extended reasoning may increase usage.

NOTES:

INTERNAL USE ONLY

APPLY IT: LLM OR AGENT?

Draft two simple workplace scenarios that could be supported by an LLM and an Agent. The scenarios should describe real work tasks or challenges that you regularly encounter and include what you're trying to accomplish.

Prompt Copilot with `/agent` to see a list of available agents.

Layer	Your Scenario
LLM	Check the local system for a given file
Agent	With the understanding the AI will act as a developer, do something...

SKILL

A **Skill** is designed for a known, repeatable output. Rather than starting from scratch, a Skill asks questions and follows set steps, applies standards, and helps produce a consistent artifact. Access available skills using the following command: `/skill`

Category	Details
Key Capabilities	Asks questions, follows set steps, applies standards, can automate repeatable tasks.
Concrete Example	Use the <code>/tdd-sentinel</code> skill to guide test-first development
Best Used When	The task is repeatable and should produce a consistent format or checklist-driven output.
Don't Use When	The task is highly unique, experimental, or unlikely to be reused.
Token Severity	Moderate
What Drives Token Use	Guided instructions, questions, review steps, and any draft revisions.
AI Credit Takeaway	Moderate cost but can save credits by reducing rework and unnecessary regenerations.

NOTES:

MCP

A Model Context Protocol (MCP) enables Agents to retrieve current information from connected systems such as Jira, Confluence, GitHub, or other approved sources. MCPs are best used when a task requires real-time data or context that is not available within the current chat.

Category	Details
Key Capabilities	Submit a Pull Request, connect to Dynatrace and investigate a problem, complete a Root Cause Analysis, pulling details from Jira, etc.
Concrete Example	Ask an Agent to access Jira and retrieve Epic details from a project and generate an implementation plan based on that information.
Best Used When	The task requires connected data or current information from approved systems.
Don't Use When	The task can be answered from the current chat, pasted text, or general reasoning alone.
Token Severity	Very high
What Drives Token Use	Large upfront initialization cost per enabled MCP before task work begins. This cost applies to every new window and/or session. Examples: GitHub ~4.9k, Jira ~1.2k, Confluence ~644 tokens. This is per session/new window.
AI Credit Takeaway	Very high cost. Multiple MCPs can add thousands of tokens upfront; unnecessary MCPs can waste credits fast.

NOTES:

INTERNAL USE ONLY

APPLY IT: SKILL OR MCP?

Draft a simple workplace scenario that could be supported by either a Skill or an MCP. Prompt Copilot with `/skill list` to see a list of available skills. You only need to write one scenario, but you may write both if helpful.

Layer	Your Scenario
Skill	Write code block that does X
MCP	Connect to external system (e.g., Jira)

CHOOSING THE RIGHT LAYER

When deciding which layer to use, ask yourself these questions:

Question	Suggested Layer
Can I accomplish this with a direct prompt?	If yes, use the LLM.
Do I need a known, repeatable artifact?	If yes, use a Skill.
Do I need a specialist to work through a broader, multi-step goal?	If yes, use an Agent.
Does the AI need access to real external system data?	If yes, enable only the MCP server or servers needed for that task.

NOTES:

GUIDED PLAYGROUND

Working with a partner, follow the prompts in the table below, working from top to bottom. Each prompt introduces a different layer of AI and builds on the previous one. Enter the prompt as written, review the response generated by AI, compare the result to the previous layer you used, and **record your observations in the last column**. Focus on what additional capabilities each layer provides and how the outputs change as more functionality is added.

NOTE: Ensure you are using the CLI. To disable/enable MCPs: `/mcp [disable/enable] uwm-[name of MCP]-mcp`. E.g., `/mcp disable uwm-Jira-mcp`

Layer	Prompt	Progression	How Was This Different Than the Previous Layer?
LLM	<i>What color is the sky?</i>	Start with a simple question that can be answered directly.	(N/A - This is the first layer.)
Agent	<i>Find a reliable source that explains why the sky appears blue. Summarize the source in one sentence and provide a link to the source.</i>	Build on the original question by asking AI to research and gather supporting information.	It's specifying a defined context or understanding the LLM is to function within
Skill	<i>Use the /uwm-developer-tooling:brainstorming skill to help plan a space-themed company event for UWM. Do not do internet research.</i>	Move from answering questions to completing a repeatable task using a structured process.	This creates either an output artifact and / or a repeatable action to accomplish a reoccurring task
Agent + Skill	(Clear the chat with /new.) <i>Use the /uwm-developer-tooling:brainstorming skill to help me write a Fibonacci function for an enterprise application. Include business use cases, coding considerations, testing considerations, and risks that should be reviewed.</i>	Combine layers to accomplish a larger goal, even when the subject matter is unfamiliar.	This combines the above two levels into a single contextual session (regardless if it's human-monitored or not)
MCP	Enable the required MCPs: <code>/mcp [disable/enable] uwm-[name of MCP]-mcp</code> <i>Using connected sources, look for related Confluence documentation or linked context that may help explain this Jira story." (Provide a Jira story of your own for this)</i>	Move beyond AI-generated knowledge and access information from connected systems.	This differs from the above entries as it is, for example, an API interact that's needed by the skill and / or agent (whom are resting on the LLM) to accomplish a desired outcome (e.g., searching for something in Jira)

REFLECTION

Think about one task where you may be using AI out of habit today and one place where you could be more intentional going forward.

Reflection Prompt	Your Response
Where might I be using AI out of habit?	Reviewing logs from failed programmatic build session
Where could I be more intentional going forward?	Use Copilot (not Copilot pulling from Claude) to minimize token utilization

FINAL THOUGHTS

Look back at your first response from the beginning of class. Would you choose the same AI layer now, or would you choose a different one?

Question	Your Response
Would you choose the same layer now?	Yes only because I have recently shifted "up" the layer hierarchy in that I infrequently leverage an agent and instead am primarily and directly interacting with the LLM
If different, which layer is the better fit?	LLM (no longer agent)
Why?	I do not need the highly-focused attention and / or detail of the agent and am able to achieve the desired outcome using the less-focused approach brought by the LLM

PLUGIN TIPS

A few helpful commands for working with Copilot plugins:

Need To...	Command
Update an installed plugin	<code>/plugin update [name-of-plugin]</code> e.g., <code>/plugin update uwm-core-tooling</code>
Install a plugin	<code>/plugin install uwm-copilot-tooling</code>

**If you have any questions or concerns,
please reach out to ittraining@uwm.com.**